Assignment 7.2: Clustering Sales Data Using K-Means

Name: Umang Patel
Course: AIT 620: Methods of Artificial Intelligence and Machine Learning
Professor: Dr. Fuat Akal
Date: October 15, 2026

Activity 5.01: In this activity I use the Sales Transactions Dataset Weekly. It has the weekly sales of 811 products over one year (52 weeks). There is no other information about the products, only their sales. I use K-Means to group products that have similar sales patterns, and then I look at which groups sell the most.

Load the data

In [1]:
```
import pandas as pd

# show only 8 columns of wide tables so they fit on the page
pd.set_option('display.max_columns', 8)

# link to the weekly sales dataset
file_url = 'https://raw.githubusercontent.com/PacktWorkshops/The-Applied-Artificial-Intelligence-Workshop/master/Datasets/Sales_Transactions_Dataset_Weekly.csv'

# read the csv file into a DataFrame
df = pd.read_csv(file_url)
df
```

Out[1]:

	Product_Code	W0	W1	W2	...	Normalized 48	Normalized 49	Normalized 50	Normalized 51
0	P1	11	12	10	...	0.22	0.17	0.11	0.39
1	P2	7	6	3	...	0.50	0.10	0.60	0.00
2	P3	7	11	8	...	1.00	0.45	0.45	0.36
3	P4	12	8	13	...	0.71	0.35	0.29	0.35
4	P5	8	5	13	...	0.13	0.53	0.33	0.40
...	...	...	...	...	...	...	...	...	...
806	P815	0	0	1	...	0.00	0.00	0.67	0.00
807	P816	0	1	0	...	0.71	0.71	0.86	0.71
808	P817	1	0	0	...	0.00	0.00	1.00	0.75
809	P818	0	0	0	...	0.00	0.00	1.00	0.00
810	P819	0	1	0	...	0.00	0.00	0.00	0.33

811 rows × 107 columns

Each row is one product. The columns are:

- **Product_Code** – the name of the product (P1, P2, ...)
- **W0 to W51** – how many units were sold in each week
- **MIN** and **MAX** – the lowest and highest weekly sales
- **Normalized 0 to Normalized 51** – the weekly sales scaled between 0 and 1

The normalized columns are better for clustering, because they put every product on the same scale. That way the products are grouped by the **shape** of their sales over the year, and not only by how big their sales are.

Keep only the normalized columns

The first 55 columns are Product_Code, W0 to W51, MIN and MAX. I drop them and keep only the 52 normalized columns for the model.

In [2]:
```
# drop the first 55 columns and keep the normalized ones
df2 = df.drop(df.iloc[:, 0:55], inplace=False, axis=1)
df2
```

Out[2]:

	Normalized 0	Normalized 1	Normalized 2	Normalized 3	...	Normalized 48	Normalized 49	Normalized 50	Normalized 51
0	0.44	0.50	0.39	0.28	...	0.22	0.17	0.11	0.39
1	0.70	0.60	0.30	0.20	...	0.50	0.10	0.60	0.00
2	0.36	0.73	0.45	0.55	...	1.00	0.45	0.45	0.36
3	0.59	0.35	0.65	0.18	...	0.71	0.35	0.29	0.35
4	0.33	0.13	0.67	0.53	...	0.13	0.53	0.33	0.40
...	...	...	...	...	...	...	...	...	...
806	0.00	0.00	0.33	0.00	...	0.00	0.00	0.67	0.00
807	0.00	0.14	0.00	0.00	...	0.71	0.71	0.86	0.71
808	0.25	0.00	0.00	0.00	...	0.00	0.00	1.00	0.75
809	0.00	0.00	0.00	0.50	...	0.00	0.00	1.00	0.00
810	0.00	0.33	0.00	0.00	...	0.00	0.00	0.00	0.33

811 rows × 52 columns

Build the K-Means model

K-Means splits the products into a set number of groups (clusters). Products in the same cluster have similar weekly sales. I use 8 clusters, like in the book, and set `random_state=8` so I get the same result every time I run it.

In [3]:
```
from sklearn.cluster import KMeans

# create the model with 8 clusters and train it on the normalized data
```

```
k_means_model = KMeans(n_clusters=8, random_state=8)
k_means_model.fit(df2)

# the cluster number given to each product
labels = k_means_model.labels_
labels
```

Out[3]:
```
array([6, 6, 2, 6, 6, 1, 6, 6, 2, 2, 6, 2, 5, 2, 5, 5, 5, 5, 5, 5, 5, 5,
       5, 5, 6, 2, 5, 5, 6, 5, 5, 5, 2, 5, 5, 6, 6, 6, 6, 5, 5, 5, 6, 5,
       6, 5, 6, 5, 5, 6, 5, 5, 2, 5, 6, 5, 5, 5, 5, 5, 5, 5, 5, 6, 2, 5,
       5, 6, 5, 5, 5, 5, 5, 6, 6, 5, 2, 5, 6, 6, 2, 6, 6, 6, 5, 5, 5, 5,
       5, 5, 2, 5, 5, 1, 2, 6, 5, 2, 6, 4, 5, 5, 5, 6, 6, 1, 2, 4, 6, 6,
       4, 6, 5, 5, 5, 2, 6, 6, 5, 5, 6, 6, 2, 6, 6, 6, 6, 5, 5, 5, 5, 6,
       5, 6, 5, 5, 5, 6, 5, 5, 5, 5, 5, 2, 6, 6, 5, 6, 5, 2, 6, 2, 6, 2,
       6, 6, 6, 6, 6, 2, 6, 6, 6, 2, 5, 6, 5, 6, 6, 5, 5, 5, 5, 5, 5, 5,
       6, 5, 5, 5, 5, 5, 6, 5, 5, 5, 5, 5, 6, 5, 5, 5, 6, 5, 6, 5, 2, 1,
       2, 1, 6, 2, 2, 2, 4, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 3, 7, 3, 7, 3,
       7, 3, 3, 7, 7, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 3, 3, 7, 3, 3, 7, 7,
       3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 7, 3, 7, 3, 3, 3, 2, 1, 4, 2,
       1, 2, 2, 1, 1, 1, 4, 3, 3, 3, 3, 3, 7, 3, 3, 3, 4, 7, 3, 5, 1, 1,
       7, 4, 0, 3, 6, 4, 6, 6, 1, 2, 6, 2, 2, 2, 6, 2, 5, 6, 6, 6, 4, 6,
       5, 5, 6, 6, 6, 2, 4, 6, 5, 6, 2, 6, 6, 2, 2, 4, 6, 6, 6, 2, 6, 1,
       6, 5, 5, 6, 6, 6, 6, 4, 4, 7, 2, 1, 4, 4, 4, 7, 7, 7, 7, 3, 3, 3,
       3, 3, 1, 1, 7, 1, 4, 2, 4, 4, 1, 4, 4, 1, 1, 1, 4, 4, 4, 7, 7, 7,
       4, 7, 7, 3, 3, 3, 3, 3, 7, 3, 4, 4, 1, 6, 4, 1, 7, 4, 4, 1, 1, 1,
       1, 1, 1, 6, 2, 2, 1, 1, 2, 1, 1, 1, 4, 6, 1, 4, 2, 7, 4, 3, 3, 3,
       3, 7, 4, 7, 3, 3, 3, 3, 1, 1, 1, 1, 7, 4, 2, 4, 2, 3, 7, 4, 7, 7,
       0, 3, 3, 3, 3, 7, 3, 3, 3, 3, 3, 0, 7, 7, 3, 3, 3, 7, 3, 7, 3, 3,
       7, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 2, 7, 4, 7, 7, 7, 4, 7, 7, 6, 2, 2,
       2, 6, 5, 1, 6, 2, 1, 4, 1, 1, 1, 6, 2, 6, 2, 2, 1, 2, 2, 2, 1, 2,
       4, 1, 6, 1, 5, 1, 2, 2, 2, 2, 5, 2, 6, 2, 2, 6, 5, 4, 6, 2, 5, 2,
       2, 2, 2, 1, 1, 2, 2, 4, 2, 5, 2, 1, 5, 5, 2, 2, 2, 6, 5, 6, 5, 5,
       6, 6, 5, 1, 6, 2, 2, 2, 6, 6, 6, 4, 2, 2, 6, 4, 6, 4, 1, 4, 4, 0,
       7, 3, 7, 3, 7, 2, 6, 6, 2, 2, 1, 4, 1, 1, 7, 1, 1, 7, 2, 1, 4, 1,
       7, 1, 7, 7, 4, 7, 7, 3, 7, 7, 7, 3, 1, 4, 4, 1, 1, 1, 1, 1, 6, 5,
       6, 5, 6, 5, 5, 5, 6, 6, 6, 5, 6, 4, 6, 2, 1, 5, 6, 5, 1, 1, 3, 1,
       6, 1, 3, 3, 3, 3, 3, 3, 7, 3, 3, 3, 3, 3, 7, 3, 3, 3, 3, 3, 7, 3,
       3, 3, 7, 3, 3, 7, 7, 3, 4, 4, 1, 4, 3, 3, 3, 3, 3, 3, 3, 3, 3, 1,
       7, 7, 3, 3, 3, 1, 1, 7, 3, 3, 7, 4, 7, 7, 4, 4, 4, 7, 1, 7, 7, 3,
       3, 3, 3, 3, 0, 3, 0, 3, 3, 3, 3, 3, 3, 7, 7, 7, 3, 3, 3, 7, 7, 7,
       3, 7, 4, 7, 7, 7, 7, 7, 7, 0, 4, 7, 4, 7, 7, 3, 3, 1, 4, 4, 7, 7,
       7, 3, 3, 3, 3, 4, 0, 4, 7, 3, 7, 4, 7, 7, 3, 3, 3, 3, 3, 7, 3, 3,
       3, 3, 1, 2, 1, 1, 1, 6, 7, 4, 4, 4, 1, 4, 4, 7, 7, 4, 4, 4, 1, 7,
       4, 7, 1, 1, 6, 4, 7, 7, 7, 3, 6, 4, 4, 4, 3, 4, 7, 7, 3],
      dtype=int32)
```

Add the cluster labels to the sales data

To see how much each cluster sells, I go back to the real weekly sales (W0 to W51). I drop the MIN, MAX and normalized columns and the Product_Code, then add the cluster label to each product.

In [4]:
```
# keep only the weekly sales columns W0 to W51
df.drop(df.iloc[:, 53:], inplace=True, axis=1)
```

```
df.drop('Product_Code', inplace=True, axis=1)

# add the cluster label for each product
df['label'] = labels
df
```

Out[4]:

	W0	W1	W2	W3	...	W49	W50	W51	label
0	11	12	10	8	...	6	5	10	6
1	7	6	3	2	...	1	6	0	6
2	7	11	8	9	...	8	8	7	2
3	12	8	13	5	...	8	7	8	6
4	8	5	13	11	...	11	8	9	6
...	...	...	...	...	...	...	...	...	...
806	0	0	1	0	...	0	2	0	3
807	0	1	0	0	...	5	6	5	4
808	1	0	0	0	...	0	4	3	7
809	0	0	0	1	...	0	2	0	7
810	0	1	0	0	...	0	0	1	3

811 rows × 53 columns

Compare the clusters

For each cluster I find:

- **count_product** – how many products are in the cluster
- **total_sales** – the total units sold by all products in the cluster over the year
- **yearly_average_sales** – the average yearly sales for one product in the cluster

In [5]:

```
# total weekly sales for each cluster
df_agg = df.groupby('label').sum()

# number of products in each cluster
df_final = df[['label', 'W0']].groupby('label').count()
df_final = df_final.rename(columns={'W0': 'count_product'})

# total sales and average yearly sales per product
df_final['total_sales'] = df_agg.sum(axis=1)
df_final['yearly_average_sales'] = df_final['total_sales'] /
df_final['count_product']

# sort from the best selling cluster to the lowest
df_final.sort_values(by='yearly_average_sales', ascending=False,
inplace=True)
df_final
```

Out[5]:

label	count_product	total_sales	yearly_average_sales
5	128	173808	1357.875000
6	129	86341	669.310078
2	96	48778	508.104167
1	90	43112	479.022222
4	85	17390	204.588235
7	107	4348	40.635514
0	8	96	12.000000
3	168	1414	8.416667

Results

The table shows that the 8 clusters are very different from each other:

- **Cluster 5** is the best selling group. Its 128 products sold about **1,358 units each** over the year, and together they sold 173,808 units. That is almost half (about 46%) of all the sales in the dataset.
- **Clusters 6, 2 and 1** are the middle groups. Their products sold between about **480 and 670 units each** per year.
- **Cluster 4** sells much less, about **205 units** per product.
- **Clusters 7, 0 and 3** sell very little. Cluster 3 is the biggest group with 168 products, but each of them sold only about **8 units** in the whole year.

Conclusion

K-Means was able to group the 811 products into 8 clusters using only their weekly sales. Even though we know nothing else about the products, the clusters clearly separate the best sellers from the products that almost never sell.

A store could use this to make better decisions. For example, it should keep plenty of stock for the products in cluster 5, since they sell the most. It could look at the products in clusters 7, 0 and 3 and think about running promotions or removing them, because they take up space but bring in very few sales.

The cluster numbers themselves (0 to 7) do not have a meaning. They are only names for the groups. Also, the exact cluster numbers and sizes can change a little with a different version of scikit-learn or a different `random_state`, but the overall pattern stays the same.